Raíz Viva – Ecosistema de automatización con IA

Entrega final

Introducción

Para este proyecto inventé un negocio: Raíz Viva, un vivero online que vende plantas, macetas y sustratos, y hace envíos. Todos los datos que usé son ficticios.

El problema que quise resolver es que un negocio así recibe muchas consultas de tipos muy distintos: gente que pregunta si hay una planta, paisajistas que quieren comprar por mayor, clientes con reclamos y también spam. Responder todo a mano lleva tiempo y es fácil que un pedido grande se pierda entre los mensajes.

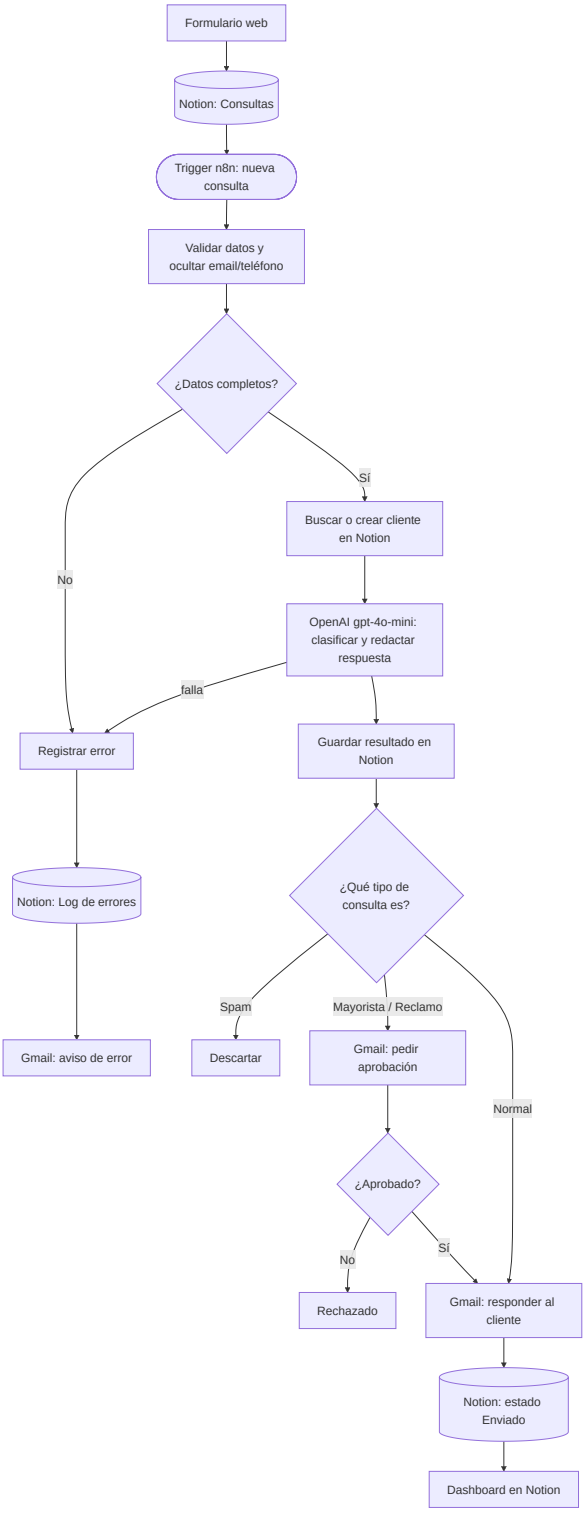
Mi solución fue armar un flujo en n8n. Las consultas llegan a una base de datos en Notion, la IA de OpenAI las clasifica y redacta una respuesta, y Gmail se encarga de enviarla. Las respuestas simples salen solas, pero las delicadas (pedidos mayoristas y reclamos) esperan a que yo las apruebe.

Herramientas que usé:

- Orquestador: n8n Cloud
- Base de datos: Notion (3 tablas relacionadas)
- IA: OpenAI, modelo gpt-4o-mini
- Canal de salida: Gmail

1. Mapa de arquitectura del sistema

Este es el diagrama del flujo completo, desde que entra la consulta hasta que se responde o se registra un error:



Partes del sistema

Parte	Qué usé
Trigger de entrada	Un trigger de Notion que revisa cada minuto si hay una consulta nueva. Solo toma las páginas nuevas, no las que se editan.
Decisiones (routers)	¿La consulta ya fue procesada?, ¿tiene los datos completos?, ¿el cliente ya existe?, ¿la respuesta de la IA es válida?, ¿qué tipo de consulta es? y ¿fue aprobada?
APIs	Notion (leer y guardar datos), OpenAI (clasificar y redactar) y Gmail (responder, pedir aprobación y avisar errores).
IA	gpt-4o-mini, configurado para que siempre devuelva un JSON.
Destino final	El email al cliente y el registro de todo en Notion.

Nodos del flujo principal

Nodo	Para qué sirve
Trigger · Nueva consulta en Notion	Arranca el flujo cuando hay una consulta nueva.
Configuración	Guarda las variables: nombre del negocio, mi email, modelo de IA, máximo de tokens, cantidad de intentos e IDs de las tablas. Así no hay datos escritos a mano en los demás nodos.
Normalizar y minimizar datos	Lee los datos de la consulta, revisa el email y el mensaje, y tapa los emails y teléfonos antes de mandar el texto a la IA.
¿Procesable? y ¿Datos completos?	Evitan que una consulta se procese dos veces y frenan las que vienen incompletas.
Buscar / crear cliente y vincular	Si el cliente no existe lo crea, y lo relaciona con la consulta.
Construir prompt, IA y Parsear respuesta	Arman el prompt con las variables, llaman a OpenAI y revisan que la respuesta tenga el formato correcto.
Guardar resultado y Router	Guardan la clasificación en Notion y deciden el camino: spam, aprobación o respuesta automática.
Nodos de aprobación (Gmail)	Me mandan un mail con botones para aprobar o rechazar, y esperan mi respuesta.
Responder al cliente y Marcar Enviado	Envían el mail y actualizan el estado en Notion.
Nodos de error	Registran la falla, marcan la consulta como Error y me avisan por mail.

2. Manual operativo de estructuras de datos

La base de datos está en Notion y tiene tres tablas relacionadas entre sí: Clientes, Consultas y Log de errores. Para armar la estructura le di instrucciones a una IA (Claude, conectado a Notion), de la misma manera que en clase lo hicimos con Omni AI en Airtable. Este fue el pedido:

```
"Creá en Notion la memoria de un sistema de atención al cliente para un vivero online
ficticio
llamado Raíz Viva. Necesito 3 bases relacionadas: Clientes (sin repetir por email), Consultas
(con estados Pendiente, Procesado por IA, Esperando aprobación, Aprobado por Humano y
Enviado,
más Error, Rechazado y Descartado; categoría, prioridad y sentimiento asignados por IA,
respuesta propuesta, tokens, modelo, Thread ID de Gmail y un contador de intentos) y Log de
errores relacionado con Consultas."
```

Tabla Consultas

Es la tabla principal. Tiene estos campos:

Campo	Tipo	Quién lo completa
Asunto, Nombre del cliente, Email remitente, Teléfono, Mensaje	Texto / email / teléfono	El cliente, con el formulario
Estado	Selección: Pendiente, Procesado por IA, Esperando aprobación, Aprobado por Humano, Rechazado por Humano, Enviado, Error o Descartado (spam)	n8n o yo
Categoría, Prioridad, Sentimiento	Selección	La IA
Resumen IA, Respuesta propuesta	Texto	La IA
Requiere aprobación	Casilla	n8n
Modelo IA, Tokens usados	Selección / número	n8n
Thread ID, Fecha de respuesta	Texto / fecha	n8n
Intentos	Número	n8n (para evitar bucles)
Cliente	Relación con Clientes	n8n
Errores	Relación con Log de errores	n8n
Éxito (%) y Error (%)	Fórmula	Notion

Tabla Clientes	
Nombre	Título
Email	Se usa para no repetir clientes
Teléfono	Con formato +54
Tipo de cliente	Minorista, Mayorista, Paisajista o Desconocido
Consultas	Relación con Consultas

Tabla Log de errores	
Error	Título
Tipo	Datos faltantes, API de IA, Respuesta IA inválida, Notion, Gmail
Nodo, Detalle, Ejecución	Texto
Consulta	Relación con Consultas
Resuelto, Fecha	Casilla / fecha

Esquemas JSON de las integraciones

Estos son ejemplos de los datos que pasan entre las herramientas (los recorté para que se lean mejor).

1. Lo que manda Notion al trigger:

```
{
  "id": "1a2b…",
  "properties": {
    "Asunto": { "type": "title",
      "title": [{ "plain_text": "¿Tienen
potus?" }] },
    "Email remitente": { "type": "email",
      "email": "lucia@ejemplo.com" },
    "Mensaje": { "type": "rich_text",
      "rich_text": [{ "plain_text": "Hola…"
}] },
    "Estado": { "type": "select", "select":
null }
  }
}
```

2. Lo que n8n le manda a OpenAI:

```
{
  "model": "gpt-4o-mini",
  "max_tokens": 500,
  "temperature": 0.3,
  "response_format": { "type": "json_object"
},
  "messages": [
    { "role": "system", "content": "Sos el
asistente
      de Raíz Viva… devolvé SOLO JSON…" },
    { "role": "user", "content": "Asunto: …
      Mensaje: … escribime a [EMAIL]" }
  ]
}
```

3. Lo que responde la IA (Message.Content):

```
{
  "categoria": "Pedido mayorista",
  "prioridad": "Alta",
  "sentimiento": "Positivo",
  "resumen": "Paisajista pide 200
suculentas",
  "respuesta": "Hola, gracias por
escribirnos…"
}
```

4. La respuesta del mail de aprobación:

```
{ "data": { "approved": true } }
```

5. Lo que se guarda en Notion al enviar:

```
{ "Estado": "Enviado",
  "Thread ID": "18f…",
  "Fecha de respuesta": "2026-11-07T22:32" }
```

6. Lo que se guarda cuando hay un error:

```
{ "titulo": "Falla de la API de OpenAI",
  "tipo": "API de IA",
  "nodo": "IA · Clasificar y redactar",
  "detalle": "The model does not exist…" }
```

3. Estrategia de optimización de costos y recursos

Comparación de modelos

Comparé cuatro modelos con los precios que encontré en noviembre de 2026, en dólares por millón de tokens. Para estimar el costo por consulta calculé unos 650 tokens de entrada (el prompt más el mensaje) y unos 250 de salida.

Modelo	Entrada / 1M	Salida / 1M	Costo por consulta	1.000 consultas	Para qué sirve
gpt-4o-mini (elegido)	0,15	0,60	≈ 0,00025	≈ 0,25	Clasificar y respuestas cortas
gpt-4o	2,50	10,00	≈ 0,0041	≈ 4,13	Tareas más complejas
Claude Haiku 4.5	1,00	5,00	≈ 0,0019	≈ 1,90	Respuestas rápidas
Claude Sonnet 4.5	3,00	15,00	≈ 0,0057	≈ 5,70	Leer textos largos y entender matices

Matriz de decisión

Tipo de tarea	Ejemplo	Qué uso	Por qué
Tareas mecánicas	Clasificar consultas y escribir respuestas cortas	gpt-4o-mini	Es una tarea sencilla y repetitiva. Este modelo alcanza y es unas 16 veces más barato que gpt-4o.
Lectura densa	Reclamos muy largos o listas de pedidos mayoristas	Claude Sonnet (a futuro)	Entiende mejor los textos largos. Lo dejé planteado como una mejora para usar solo en mensajes de más de 1.500 caracteres, que son pocos, así que casi no sube el costo.
Procesos masivos	Un informe mensual con todas las consultas para encontrar las preguntas más frecuentes	API de lotes (Batch)	No necesita respuesta inmediata, así que se puede mandar en lote y cuesta la mitad.

Cálculo del ahorro con Batch: si el informe mensual analiza 1.000 consultas (unos 1.500 tokens de entrada y 300 de salida cada una) con Claude Sonnet, en tiempo real costaría 1.000 × (1.500 × 3 + 300 × 15) / 1.000.000 = USD 9,00. Con la API de lotes cuesta la mitad: USD 4,50, o sea un 50% de ahorro.

Otras cosas que hice para gastar menos

- Limité las respuestas a 500 tokens y le pedí a la IA que no pase de 120 palabras.
- A la IA solo le mando el asunto y el mensaje, recortado a 2.000 caracteres.
- Si los datos están incompletos, la consulta no llega a la IA, así que no se gasta nada.
- El trigger solo toma consultas nuevas y cada consulta puede intentarse como máximo 3 veces.
- Guardo los tokens usados en cada consulta para poder controlar el gasto.

4. Malla de seguridad, privacidad y resiliencia

Cuidado de los datos personales

- Minimización: la IA no recibe el email ni el teléfono del cliente, solo el asunto, el mensaje y el primer nombre para el saludo.
- Anonimización: si el cliente escribe su email o su teléfono dentro del mensaje, el flujo los cambia por [EMAIL] y [TEL] antes de mandarlo a la IA.
- Trazabilidad: en Notion queda registrado qué modelo se usó, cuántos tokens, el estado y la fecha de cada consulta. Los errores quedan con el nodo donde fallaron.
- Retención: propongo borrar el spam después de 30 días y revisar una vez al año los clientes que ya no compran, teniendo en cuenta la Ley 25.326 de Protección de Datos Personales.
- Claves: las API keys están guardadas en las credenciales de n8n y no aparecen en los archivos que subí.

Rutas de error

En n8n configuré los errores con la opción "On Error". Cuando elijo "Continue (using error output)", el flujo sigue por otro camino, como el Resume de Make. Cuando elijo "Stop workflow", el flujo se corta, como el Break.

Qué puede fallar	Cómo lo resolví
Faltan datos (email mal escrito o mensaje muy corto)	No se llama a la IA. Se registra en el Log de errores, la consulta pasa a Error y me llega un mail.
Falla la API de OpenAI	El nodo reintenta 3 veces. Si sigue fallando, sale por la salida de error, se registra y me avisa.
La IA devuelve algo con mal formato	Un nodo de código revisa la respuesta. Si no es válida, se registra el error y no se le manda nada al cliente.
No se puede enviar el mail	Se registra el error y la consulta queda en Error para responderla a mano.
Cualquier otro error	Hay un segundo flujo (Manejador global de errores) que se activa si el principal se corta y me manda una alerta.

Aprobación humana (human-in-the-loop)

Para que la IA no le conteste cualquier cosa a un cliente en una situación delicada, el flujo se frena antes de responder en estos casos:

- Pedidos mayoristas, porque tienen precios y cantidades que la IA no debería prometer.
- Reclamos, porque una mala respuesta puede empeorar el problema (y está en juego la Ley de Defensa del Consumidor).
- Mensajes con sentimiento negativo o prioridad alta.

En esos casos me llega un mail con el mensaje del cliente, la respuesta que propone la IA y dos botones: "Aprobar y enviar" y "Rechazar". El flujo queda esperando hasta que elijo uno. Además, en el prompt le pedí a la IA que no invente precios, stock ni plazos.

Revisión de seguridad

- ¿Hay un filtro contra bucles infinitos? Sí: el trigger solo toma consultas nuevas, se revisa que el estado esté vacío o en Pendiente y hay un contador de intentos con un máximo de 3.
- ¿Se comparan bien los tipos de datos? Sí: los filtros comparan booleano con booleano y número con número, con la validación estricta activada.
- ¿El prompt es dinámico? Sí: usa las variables del nodo Configuración y los datos de cada consulta.

5. Dashboard de Control Ejecutivo

El dashboard es la misma página de Notion, publicada en la web en modo solo lectura. Como con el plan gratis de Notion no pude usar gráficos, armé una tabla de consultas agrupada por estado. Notion muestra cuántas consultas hay en cada grupo, y con eso se ven los indicadores:

Indicador	Resultado con mis pruebas
Consultas totales	6
Tasa de éxito (consultas enviadas)	2 de 6 (33%)
Tasa de error	2 de 6 (33%), las dos provocadas a propósito en las pruebas
Consultas que pasaron por aprobación humana	2 (una aprobada y una rechazada)
Spam descartado	1

Link al dashboard: crawling-cayenne-74e.notion.site/Ra-z-Viva-Ecosistema-de-Automatizaci-n-IA

6. Pruebas

Ejecuté el flujo 6 veces probando distintos casos, incluidos dos "caminos infelices":

#	Caso	Qué pasó
1	Consulta común: "¿Tienen potus?"	Se clasificó como venta minorista y se respondió automáticamente.
2	Paisajista que pide 200 suculentas	Pedido mayorista: me llegó el mail de aprobación, lo aprobé y se envió la respuesta.
3	Reclamo por una maceta rota	Lo rechacé y no se le mandó nada al cliente.
4	Email "juan@" y mensaje "hola"	No pasó la validación. Se registró como datos faltantes y me llegó la alerta.
5	Modelo de IA que no existe	Hizo 3 reintentos, falló, se registró el error y me llegó la alerta.
6	Mensaje de spam	Quedó descartado y sin respuesta.

Como mejoras a futuro, se podría usar Claude para los mensajes muy largos, pasar la aprobación a Slack, sumar WhatsApp como canal (con plantillas aprobadas y números con formato +54) y armar el informe mensual con la API de lotes.